Informatics Institute of Technology

Department of Computing

In collaboration with

University of Westminster

Individual Report

5COSC022C.2

Client-Server Architectures

Name: Isuru Udara
IIT ID: 20240906
UOW ID: w2120151

Part 1: Service Architecture & Setup

1. Project & Application Configuration

Question: In your report, explain the default lifecycle of a JAX-RS Resource class. Is a new instance instantiated for every incoming request, or does the runtime treat it as a singleton? Elaborate on how this architectural decision impacts the way you manage and synchronize your in-memory data structures (maps/lists) to prevent data loss or race conditions .

The Lifecycle
By default, JAX-RS uses a "per request" lifecycle. This means the server creates a brand new instance of a resource class (like RoomResource) for every single HTTP request, and then destroys it immediately after sending the response.

Managing State
Because these resource instances are destroyed so quickly, data cannot be saved directly inside them. If you stored a list of rooms in RoomResource, that list would be wiped clean on every single request. To solve this, the project uses a Singleton pattern via a DataStore class. This creates one permanent, shared "safe" for the data. All the temporary, per-request resource classes read and write to this single DataStore instance, preventing data loss.

```
private static final DataStore instance = new DataStore();
public static DataStore getInstance() { return instance; }
```

Thread Safety Considerations
While the Singleton solves data loss, standard collections like HashMap are not thread-safe. If JAX-RS processes two POST requests at the exact same millisecond, concurrent writes could corrupt the map. In a production environment, we would use a ConcurrentHashMap and ArrayList with Collections.synchronizedList() or CopyOnWriteArrayList to fix this, but a standard HashMap is perfectly acceptable for the single-user testing scope .

2. The ”Discovery” Endpoint

Question: Why is the provision of ”Hypermedia” (links and navigation within responses) considered a hallmark of advanced RESTful design (HATEOAS)? How does this approach benefit client developers compared to static documentation?

HATEOAS (Hypermedia As The Engine Of Application State) means an API response doesn't just return data; it also returns navigational links telling the client what it can do next.
HATEOAS (Hypermedia As The Engine Of Application State) is considered the highest maturity level of RESTful API design. It is a hallmark because it makes the API self descriptive and self navigating. Instead of simply returning raw, dead data, the server returns the data along with hypermedia links dictating the next possible actions (e.g., providing a link to "delete" a room right next to the room's data). This shifts the control of the application's state from the client to the server.

Benefits Over Static Documentation

Relying on hypermedia offers significant advantages to client developers over traditional static documentation:

- **No Hardcoded URLs**: Developers do not need to memorize or hardcode specific endpoint paths (like /api/v1/rooms) into their applications. They simply look for the "rooms" link in the initial API response.
- **Resilience to Change**: If backend developers change the routing structure, client applications will not break. The client automatically follows the newly updated links provided by the server.
- **Always Accurate** (Live Documentation): Static documentation easily becomes outdated as a project evolves. HATEOAS acts as "live" documentation; because the server generates the links dynamically, the available actions are always 100% accurate to the current system state.

Part 2: Room Management

1. RoomResource Implementation

Question: When returning a list of rooms, what are the implications of returning only IDs versus returning the full room objects? Consider network bandwidth and client side processing.

Implications of Returning Full Objects

- **Network Bandwidth:**

Every response carries all fields for every room (id, name, capacity, sensorIds) in a single payload. At the current scale, this overhead is negligible. Even at a larger scale, because sensorIds contain short strings rather than deeply nested objects, the payload growth remains manageable.

- **Client-Side Processing:**
  Low complexity. The client receives everything it needs in a single request, requiring no follow up API calls to display room names, capacities, or sensor assignments.

Implications of Returning Only IDs

- **Network Bandwidth:**
  Returning only an array of ID strings (e.g., ["LIB-301", "LAB-101"]) produces a much lighter response regardless of scale.
- **Client-Side Processing:**
  High complexity and latency due to the N+1 Problem. The client must make a separate GET /api/v1/rooms/{roomId} request for every ID to retrieve usable data. For a list of 50 rooms, this requires 51 total HTTP requests.

The current implementation makes the pragmatic choice. Because the Room model contains a small, bounded number of fields, returning full objects in the list response is preferable. Returning only IDs is generally only justified if the objects are massive or if clients rarely need the full details of every item at once.

2. RoomDeletion & Safety Logic

Question: Is the DELETE operation idempotent in your implementation? Provide a detailed justification by describing what happens if a client mistakenly sends the exact same DELETE request for a room multiple times.

By definition, an idempotent operation must produce the same result no matter how many times it is called with the same input. No, the DELETE /api/v1/rooms/{roomId} operation in this implementation is not strictly idempotent.

Possible Outcomes on the First Call

Unlike a simple delete operation, the deleteRoom() method includes business rule preconditions, leading to three possible initial outcomes:

1. Room does not exist: Returns 404 Not Found.
2. Room exists but has sensors assigned: Throws a RoomNotEmptyException, returning 409 Conflict.
3. Room exists and has no sensors: Deleted successfully, returning 200 OK.

Behavior Across Repeated Calls

Assuming the first call succeeds (200 OK), a second identical request will find that the room no longer exists and will return a 404 Not Found. Because the HTTP response code changes between calls (from 200 to 404), the operation is not strictly idempotent from the client's perspective.

RFC 9110 and Best Practices

The HTTP specification (RFC 9110) states that DELETE should be idempotent regarding server state (the room remains deleted). While some argue that returning a 404 on a repeat DELETE is acceptable since the desired end state (the room does not exist) is met, shifting response codes can cause issues. For example, if a client experiences a network timeout and retries, receiving a 404 might cause them to incorrectly assume the initial deletion failed. To make the implementation fully idempotent, repeated DELETE calls on an already deleted resource should ideally return 204 No Content.

Part 3: Sensor Operations & Linking

1. Sensor Resource & Integrity

Question: We explicitly use the @Consumes(MediaType.APPLICATION_JSON) annotation on the POST method. Explain the technical consequences if a client attempts to send data in a different format, such as text/plain or application/xml. How does JAX-RS handle this mismatch?

The Framework Gatekeeper & HTTP 415

When a POST request is sent to /api/v1/sensors, JAX-RS inspects the Content-Type header provided by the client and attempts to match it against the @Consumes annotation. If a client sends an unsupported format (like text/plain or application/xml), JAX-RS automatically intercepts and rejects the request, returning an HTTP 415 Unsupported Media Type status code.

Technical Consequences

This rejection happens at the framework's routing stage, before the resource method (createSensor()) ever executes. The technical benefits of this are significant:

- **Zero Partial Processing:**

Because the method is never called, there is absolutely no risk of partial processing or data corruption within the DataStore.

- **Prevention of Internal Errors:**

This acts as a strict contract enforcement gate. If JAX-RS allowed a text/plain body to reach the Jackson deserializer, Jackson would fail to parse it into a Sensor object and throw an exception, resulting in an unhelpful HTTP 500 Internal Server Error. The @Consumes annotation prevents this crash entirely by validating the format upfront and returning a clean, standardized client error.

2. Filtered Retrieval & Search

Question: You implemented this filtering using @QueryParam. Contrast this with an alternative design where the type is part of the URL path (e.g., /api/v1/sensors/type/CO2). Why is the query parameter approach generally considered superior for filtering and searching collections?

In REST architecture, the URL path identifies what resource is being accessed, while query parameters describe how that resource should be filtered or modified. The @QueryParam approach (GET /api/v1/sensors?type=CO2) is vastly superior to the path-based approach (GET /api/v1/sensors/type/CO2) for several architectural reasons:

Design Concern	**@QueryParam**	**Path Segment**
Resource Identity	Correctly identifies that type=CO2 is just a filter applied to the existing /sensors collection.	Incorrectly implies that type/CO2 is a distinct, standalone sub-resource.
Optionality	Naturally optional. If omitted (/sensors), the API simply returns the unfiltered collection.	Path segments are mandatory. Supporting an unfiltered list would require building entirely separate route methods.
Composability	Easily scalable for multiple filters (e.g., ?type=CO2&status=ACTIVE).	Becomes heavily ambiguous and difficult to route (e.g., /type/CO2/status/ACTIVE).
REST Semantics	The universally accepted standard for filtering, sorting, and searching.	Should be strictly reserved for identifying unique entities (e.g., /sensors/TEMP-001).

To further improve usability, the filtering logic utilizes .equalsIgnoreCase() when evaluating the query parameter. This means ?type=co2 and ?type=CO2 will return the exact same results. This forgiving design reduces the likelihood of clients receiving empty arrays simply due to mismatched letter casing.

Part 4: Deep Nesting with Sub- Resources

Question: Discuss the architectural benefits of the Sub-Resource Locator pattern. How does delegating logic to separate classes help manage complexity in large APIs compared to defining every nested path (e.g., sensors/{id}/readings/{rid}) in one massive controller class?

Rather than defining every nested endpoint directly inside SensorResource, this implementation uses a Sub-Resource Locator method to delegate all /readings routing to a dedicated class:

```
@Path("/{sensorId}/readings")
public SensorReadingResource getReadingResource(@PathParam("sensorId") String sensorId) {
   return new SensorReadingResource(sensorId);
}
```

When JAX-RS encounters a request to /api/v1/sensors/{sensorId}/readings, it calls this method, receives the returned SensorReadingResource instance, and then scans that new class for its own @GET, @POST, and @Path annotations to complete the routing.
Delegating logic in this way provides three major architectural benefits for managing complexity:

1. The Single Responsibility Principle (SRP)

Each resource class is strictly limited to managing one entity. SensorResource manages sensors, and SensorReadingResource manages readings. If the readings logic requires modification (e.g., adding pagination or date filtering), only SensorReadingResource is touched. There is zero risk of accidentally breaking sensor creation or deletion logic because those operations are isolated in a completely separate file.

2. Controlled Context Passing

By extracting the sub-resource, the parent context (sensorId) is passed exactly once through the constructor:

```
public SensorReadingResource(String sensorId) {
   this.sensorId = sensorId;
}
```

Every endpoint method within SensorReadingResource automatically has access to the correct sensor context. Developers do not need to repeatedly declare, receive, and validate the sensorId as a @PathParam in every single method, drastically reducing boilerplate code and keeping methods focused purely on business logic.

3. Avoiding the "God Controller" Anti Pattern

Without this pattern, a single controller class would be forced to define every endpoint across all levels of the resource hierarchy:

- GET /sensors
- POST /sensors
- GET /sensors/{id}
- GET /sensors/{id}/readings
- POST /sensors/{id}/readings
- GET /sensors/{id}/readings/{rid}

As an API scales, this single class expands into thousands of lines, becoming incredibly difficult to read, maintain, and test. The Sub-Resource Locator pattern solves this by distributing responsibility across small, focused classes each responsible for exactly one level of the REST hierarchy. If new nested resources are needed in the future (e.g., /sensors/{id}/calibrations), developers simply create a new, dedicated class rather than bloating an already complex controller.

4. Independent Testability

Because SensorReadingResource is essentially a plain Java class with a constructor, it can be unit-tested in complete isolation. A test can instantiate it directly with a mock sensorId without needing to spin up the full JAX-RS runtime, configure HTTP servers, or mock routing contexts. This makes writing focused, lightweight unit tests significantly more straightforward compared to testing private or nested methods buried inside a monolithic controller.

5. Additive Scalability

If the API later needs to support additional nested sub-resources such as /sensors/{id}/alerts or /sensors/{id}/calibrations each new path simply receives its own dedicated resource class. The growth of the API becomes additive. New classes are introduced rather than an existing class being continuously expanded and modified. This keeps every individual component bounded, manageable, and easy to reason about independently.

The sub-resource locator pattern enforces a clean separation of concerns across all levels of the REST hierarchy. Rather than allowing one monolithic controller to grow indefinitely, responsibility is distributed across small, focused classes, with each responsible for exactly one level of the resource tree. This mirrors the core design principles of well-structured object-oriented software: creating small, highly focused units that are easy to understand, modify, and test independently.

Part 5: Advanced Error Handling, Exception Mapping & Logging

Question: Why is HTTP 422 often considered more semantically accurate than a standard 404 when the issue is a missing reference inside a valid JSON payload?

When a client sends a POST request to /api/v1/sensors with a valid URL and valid JSON, but includes a roomId that does not exist, the correct HTTP status to return is 422 Unprocessable Entity.

The distinction between 404 and 422 is crucial for API clarity:

- **HTTP 404 Not Found**: This is the correct response when the requested URL or endpoint cannot be located (e.g., GET /api/v1/rooms/FAKE-999). It indicates the specific resource identified by the routing path is absent.
- **HTTP 422 Unprocessable Entity**: This signals that the request was received, the URL is valid, the endpoint exists, and the JSON was successfully parsedbut the logical content of the payload is semantically invalid.

Using 404 in this scenario would be semantically misleading because the /sensors endpoint itself is perfectly valid and reachable. The issue is purely a data integrity violation within the request body. The LinkedResourceNotFoundExceptionMapper correctly returns 422 to clearly communicate that the request was understood, but the referenced entity within the payload could not be resolved.

Question: From a cybersecurity standpoint, explain the risks associated with exposing internal Java stack traces to external API consumers. What specific information could an attacker gather from such a trace?

Exposing raw Java stack traces in API responses is a significant security vulnerability, classified under OWASP A05: Security Misconfiguration. An attacker analyzing an exposed stack trace can extract several categories of highly sensitive information:

1. Technology Fingerprinting

A stack trace reveals the exact frameworks, libraries, and versions running on the server:

org.glassfish.jersey.server.ServerRuntime$Responder...
com.fasterxml.jackson.databind.exc.MismatchedInputException...

From this alone, an attacker knows the application is running Jersey and Jackson. They can immediately search for known CVEs (Common Vulnerabilities and Exposures) associated with those exact versions to identify exploitable weaknesses.

2. Internal Package and Class Structure

com.smartcampus.model.DataStore.getInstance
com.smartcampus.resource.RoomResource.deleteRoom

The attacker learns internal package names, class names, method names, and exact line numbers. This significantly lowers the effort required to craft targeted injection or exploitation attempts.

3. Business Logic Inference

Method names such as deleteRoom, createSensor, and getSensorIds expose internal workflows and data relationships that a well-designed API should keep completely opaque to external consumers.

Mitigation

The GlobalExceptionMapper mitigates all of these risks by catching every Throwable and logging full exception detail server-side only:

```
LOGGER.log(Level.SEVERE, "Unexpected error occurred", exception);
```

This ensures the complete stack trace is recorded securely in the server logs for debugging purposes, while the client receives only a safe, generic 500 Internal Server Error message revealing nothing useful to a potential attacker.

Question: Why is it advantageous to use JAX-RS filters for cross-cutting concerns like logging, rather than manually inserting Logger.info() statements inside every single resource method?

Problem	**Consequence of Manual Logging**
Code Duplication	The same log format must be copy pasted into dozens of methods. Changing the format requires editing every single file.
Human Error	A developer adding a new endpoint can easily forget to include the logging statement.
Missed Logs	If a request is rejected before reaching a resource method (e.g., a 415 Unsupported Media Type error), no manual log will ever fire.
Mixed Concerns	Business logic (e.g., creating a room) becomes cluttered with infrastructure concerns (e.g., logging).

The Filter Advantage

The LoggingFilter class solves all of these problems by implementing both ContainerRequestFilter and ContainerResponseFilter. This allows it to run automatically at the framework level for every incoming request and every outgoing response before and after any resource method executes.

This provides massive advantages:

- **Zero Additional Code:** Adding a new endpoint automatically includes logging.
- **Centralized Maintenance:** Changing the log format requires editing only one class.
- **Guaranteed Execution:** Logging is guaranteed even for requests that fail framework-level validation and never reach the business logic.
- **Separation of Concerns:** Resource classes remain focused entirely on business logic, while the filter handles observability as a clean, independent, and reusable layer.